UNIVERSITY OF MINES AND TECHNOLOGY, TARKWA

CY376: NETWORK MONITORING, SECURITY AND AUDITING

End-of-Semester Project Report

Digital Forensic Evidence Vault

and Case Management System

A Web-Based Platform for Chain-of-Custody Tracking and Tamper-Evident Audit Logging

Name:	Boakye Alfred Osei
Index Number:	FCM 41.018.099.23
Track:	Blue Team
Course Code:	CY376
GitHub Repository:	https://github.com/Alphred-OB/forensic_collection_-_Toolkit.git
Date:	3rd August 2026

Abstract

Digital evidence – files, device images, screenshots, and log exports – is fragile: easy to copy, easy to alter, and difficult to prove was handled properly after collection. Organisations that rely on paper forms, spreadsheets, and generic file storage to track evidence have no reliable way to prove a file's integrity, no trustworthy record of who handled it and when, and no enforced access control over who can view sensitive material. This project designs and implements a web-based Digital Forensic Evidence Vault and Case Management System that addresses these gaps directly. The platform, built on Laravel 11 and MySQL, provides case creation and assignment, evidence intake with automatic SHA-256 and MD5 hashing on upload, a formal chain-of-custody transfer workflow requiring explicit receiver acceptance, role-based access control across Administrator, Investigator, and Reviewer roles, TOTP-based two-factor authentication, a tamper-evident audit log built as a cryptographic hash chain, and exportable PDF chain-of-custody and case reports. Beyond building the system, this report documents a self-audit conducted against the live codebase during the final week of development, which identified and remediated a defect in report generation and catalogued a set of gaps between the original design specification and the current implementation – most notably the absence of at-rest encryption for evidence files and unresolved dependency advisories in the underlying framework. The result is a working Phase 1 platform that satisfies the project's core must-have requirements and demonstrates, in its own audit trail, the central forensic principle the course is built around: that integrity should be provable, not assumed.

Contents

1 Introduction

1.1 Background

Digital evidence occupies an unusual position among the things an organisation must manage: it is simultaneously the most important record an investigation produces and the easiest one to damage without meaning to. A single copy operation performed by the wrong tool can alter file timestamps; a screenshot taken without care can leak information it should not; a spreadsheet cell edited by the wrong person, with no log of the edit, can quietly break a chain of custody that took weeks to build. In legal and internal-review contexts, the question is rarely "did something happen to this evidence" – it is "can you prove nothing happened to it." Where that proof does not exist, the evidence itself, however genuine, can be excluded.

Most small organisations and student-scale investigative teams do not have access to enterprise digital-forensics suites, and instead default to a mix of paper chain-of-custody forms, spreadsheets, and shared folders. This is understandable – it requires no infrastructure – but it creates exactly the risks described above: no cryptographic proof of integrity, no unified and tamper-resistant record of custody, no access control beyond folder permissions, and no fast way to produce audit-ready documentation when it is needed.

1.2 Problem Statement

There is no lightweight, self-hosted system that lets a small investigative team log evidence, track who is responsible for it at every point in time, and prove – cryptographically, not just on paper – that neither the evidence nor the record of its handling has been altered. This project treats that as a solvable engineering problem rather than an unavoidable process gap.

1.3 Objectives

The project set out to:

1. Provide a single system for logging evidence, tracking custody, and generating documentation, replacing manual paper or spreadsheet processes.
2. Make every piece of evidence's integrity provable at any time using cryptographic hashing (SHA-256), computed by the server rather than trusted from the uploader.
3. Automatically and tamper-evidently log every consequential action taken on evidence – view, upload, transfer, export – without relying on manual log-keeping.
4. Enforce access based on role and case assignment, so a user can only see evidence for cases they are actually part of.
5. Produce ready-to-export chain-of-custody and case-summary reports.
6. Remain usable by a team without deep digital-forensics background: the workflow should guide correct handling rather than assume expert knowledge.

1.4 Scope

The system is a web platform only; it does not perform device acquisition. Copying data off a physical device in a forensically sound manner requires direct hardware access and write-blocking tools that a browser cannot provide – this is a structural limitation, not a design shortcut. The realistic and industry-normal division of labour is preserved: an investigator uses an existing external tool (FTK Imager, Cellebrite, Magnet AXIOM, Autopsy, or similar) to acquire a forensic image or export, and this platform's responsibility begins the moment that file is uploaded – hashing it, timestamping it, recording who uploaded it, and starting its chain of custody. Automated forensic analysis (malware analysis, artifact parsing, timeline reconstruction) is likewise out of scope: the platform manages and documents evidence, it does not analyse its contents.

1.5 Report Structure

Section 2 reviews the standards and tooling that shaped the design. Section 3 describes the development environment, system architecture, and data model. Section 4 walks through what was actually implemented. Section 5 presents results, including a security self-audit conducted against the live codebase. Section 6 analyses the gap between the original design intent and the current build, and makes prioritised recommendations. Section 7 concludes.

2 Literature and Tooling Review

2.1 Chain-of-Custody and Evidence Integrity Standards

Three reference frameworks shaped the design, chosen because together they cover the full lifecycle the platform needed to support: collection through to court-ready output.

NIST SP 800-86, *Guide to Integrating Forensic Techniques into Incident Response* (Kent, Chevalier, Grance and Dang, 2006) [1], sets out the principle that forensic soundness depends on maintaining data integrity and a documented history of handling from the point of collection onward. This directly motivated the decision to hash every file on the server side at the moment of upload, rather than trust a hash supplied by the client.

ISO/IEC 27037:2012, *Guidelines for identification, collection, acquisition, and preservation of digital evidence* [2] frames custody as something that must be actively preserved through explicit handover, not assumed to persist by default. This underpins the platform's custody-transfer model, in which responsibility only moves from one user to another once the receiving party explicitly accepts the transfer – a transfer cannot be logged unilaterally by the sender alone.

SWGDE Best Practices for Digital Evidence Collection, document 18-F-002 [3], provided the practical basis for the evidence-intake fields: source device, collection date/time/location, and an explicit classification of each item's evidentiary weight (original, forensic copy, export, screenshot, or reconstructed), so that anyone reviewing a case later understands exactly what kind of artefact they are looking at.

2.2 Cryptographic Hashing for Integrity Verification

SHA-256 was chosen as the platform's primary integrity mechanism because it is a deterministic, one-way function: the same file always produces the same fixed-length digest, and altering even a single bit produces a completely different one, with no practical way to work backward from the digest to the file. Older algorithms such as MD5 and SHA-1 are now considered too weak for integrity guarantees because practical collision attacks have been demonstrated against both; they are retained in this platform only as a secondary, compatibility-oriented value, since many external acquisition tools still report MD5 alongside SHA-256 in their own output, and cross-checking the two provides an additional sanity check at intake.

2.3 Tamper-Evident Logging

The audit log's design follows the general principle established by Schneier and Kelsey in *Secure Audit Logs to Support Computer Forensics* [4]: rather than trusting that a log has not been altered, each entry can be cryptographically linked to the one before it, so that altering or deleting any past entry breaks every hash that follows it. This turns tampering from something

that must be trusted not to have happened into something that is mechanically detectable. The platform's audit log implements this as an append-only hash chain, described in Section 4.5.

2.4 Access Control Model

Role-Based Access Control, as formalised in ANSI INCITS 359 [5], assigns permissions to roles rather than to individuals, so that what a user can see or do follows from their assigned role (Administrator, Investigator, Reviewer) rather than being configured per person. The platform layers case-level assignment on top of role, so that role alone does not grant visibility into a case – a user must additionally be assigned to that specific case, which is the mechanism that prevents cross-case evidence leakage.

2.5 Technology Stack

Table 1: Technology stack and rationale

Layer	**Choice**	**Rationale**
Backend	Laravel 11.55 (PHP 8.2) [6]	Mature MVC framework with built-in authentication scaffolding, migrations, and policy-based authorisation, reducing the amount of security-critical plumbing written from scratch.
Frontend	Blade templates, Tailwind CSS, Chart.js	Server-rendered pages avoid the additional attack surface and complexity of a separate SPA frontend; Chart.js covers the dashboard analytics requirement without a heavier charting stack.
Database	MySQL	Relational integrity constraints suit the strongly relational nature of cases, evidence, custody transfers, and audit entries.
PDF generation	barryvdh/laravel-dompdf [7]	Native Laravel integration for generating chain-of-custody and case-summary reports as real, downloadable PDF files.

Layer	**Choice**	**Rationale**
Authentication	Laravel Breeze + custom TOTP service	Breeze provides a vetted authentication baseline; a custom Time-based One-Time Password service adds two-factor authentication for privileged roles.
Local environment	XAMPP (Apache, MySQL, PHP)	Standard, low-friction local development stack suitable for a single-developer, single-machine build.

3 Methodology

3.1 Development Environment

The system was developed on a local XAMPP stack running Apache, MySQL, and PHP 8.2, with the Laravel 11.55 application managed through Composer and served via Laravel's local development server. No production infrastructure (Hostinger hosting, Cloudflare edge security, TLS termination) was provisioned during this phase; those remain deployment-stage requirements rather than something exercised during development, and are addressed as recommendations in Section 6.

3.2 System Architecture

The platform follows a standard three-layer web architecture, shown in Figure 1. The presentation layer is built from server-rendered Blade views styled with Tailwind CSS, with Chart.js handling the dashboard's analytics charts. The application layer – Laravel controllers and services – is where hashing, custody-transfer logic, permission checks, and report generation happen. The data layer combines MySQL for structured records (cases, users, evidence metadata, custody transfers, audit entries) with the local filesystem for the evidence files themselves; only file metadata and hash values are stored in the database, not the raw files.

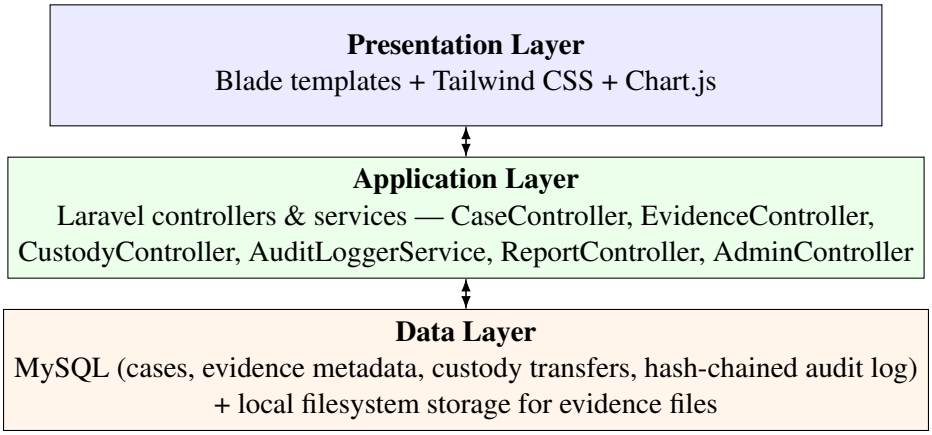

Figure 1: Three-layer system architecture

3.3 Data Model

Figure 2 shows the core entities and their relationships, drawn from the project's actual migrations. A `case` has many assigned users (via `case_assignments`, which also records each user's role within that specific case) and many evidence items. Each `evidence_item` accumulates a history of `custody_transfers` and can be referenced by one or more generated `reports`. All user activity, regardless of which entity it touches, is recorded in `audit_logs`, which is the hash-chained, insert-only table described in Section 4.5.

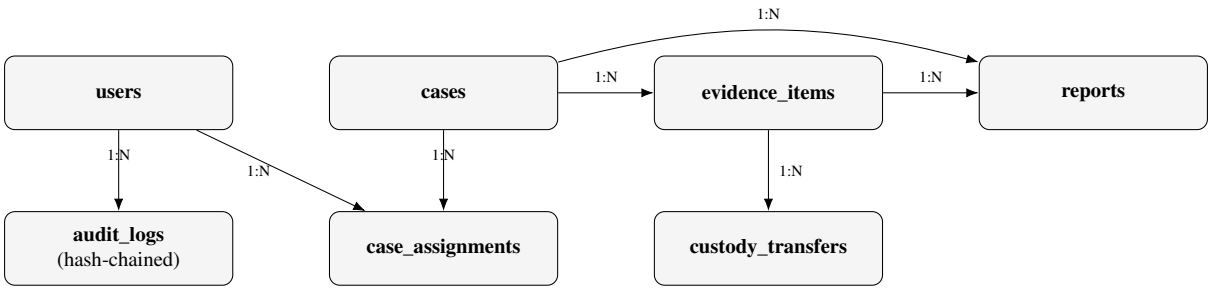

Figure 2: Core entity-relationship model

3.4 Development Process

The build proceeded in the following order: requirements analysis and persona definition (captured in the project's PRD), architecture and schema design (captured in the project's TRD), authentication and role-based access control, evidence intake with hashing, the custody-transfer workflow, the hash-chained audit log, PDF reporting, dashboard analytics, and finally a security self-audit against the completed codebase, described in Section 5.3. This last step – auditing the system after building it rather than only while designing it – is itself the practice the course is named for, and it directly produced the fix described in Section 4.6 and the gap analysis in Section 6.

4 Implementation

Table 2 summarises what is actually implemented, verified against the routes, controllers, and migrations in the codebase rather than against the original design documents' ambitions.

Table 2: Implemented feature inventory

Feature	**Location in codebase**	**Notes**
Case management (create/assign/close/archive)	`CaseController`	Soft-delete supported via `deleted_at`
Evidence intake with SHA-256 + MD5 hashing	`EvidenceController`	Classification: original / forensic_copy / export / screenshot / reconstructed
Chain-of-custody transfers	`CustodyController`	Pending → accepted/rejected; receiver must explicitly accept
Tamper-evident audit log	`AuditLoggerService`	Hash-chained, insert-only; verification routine walks the whole chain
Role-based access control	`EnsureIsAdmin` middleware + per-controller checks	Roles: Administrator, Investigator, Reviewer
Two-factor authentication (TOTP)	`TwoFactorController`, `TwoFactorService`	Setup, recovery codes, login challenge
PDF report generation	`ReportController`	Chain-of-custody and case-summary reports; each embeds a SHA-256 manifest hash
Batch evidence export	`EvidenceController::` `exportBatchZip`	ZIP export of multiple items
Dashboard analytics	Chart.js views	Case status, evidence classification, audit velocity
Case timeline / event map	Case detail view	
Hex viewer + EXIF metadata	Evidence detail view	
Case notes and notifications	`CaseNote`, `UserNotification`	
Audit log CSV export + integrity scan	`AuditController`	
Admin console	`AdminController`	User management, global evidence vault, settings

4.1 Data Model in Code

The following reflects the core schema as actually migrated:

Listing 1: Core schema (fields per table)

```
cases (case_number, title, status[open|closed|archived],
created_by, closed_at, deleted_at)

case_assignments (case_id, user_id, role_in_case)

evidence_items (case_id, evidence_number, classification,
file_path, file_hash_sha256, uploaded_by,
current_custodian_id, collected_at, collected_location)

custody_transfers (evidence_item_id, from_user_id, to_user_id,
reason, status[pending|accepted|rejected])

audit_logs (user_id, action_type, target_type, target_id,
details_json, entry_hash, previous_entry_hash)
-- insert-only hash chain

reports (case_id, evidence_item_id, type[coc_form|final_report],
file_path, manifest_hash)
```

4.2 Authentication and Access Control

Login is handled through Laravel Breeze's scaffolding, extended with a custom `TwoFactorService` that issues and validates TOTP codes; users with privileged roles must complete a secondary challenge after password authentication before a session is granted. Authorisation is enforced at the middleware and controller level – not only hidden in the interface – so that a request for a case or evidence item a user is not assigned to is rejected server-side regardless of what the UI would otherwise render.

Access control is implemented in two layers: a route-level middleware that gates administrator-only areas, and a per-case authorisation check applied inside every controller that touches case-scoped data.

Listing 2 shows the admin-only route gate.

Listing 2: `app/Http/Middleware/EnsureIsAdmin.php`

```
class EnsureIsAdmin
{
       public function handle(Request $request, Closure $next):
           Response
       {
              if (!Auth::check() || Auth::user()->role !== '
                  Administrator') {
```

```
                abort(403, 'Access␣Restricted:␣Administrator
                    ␣privileges␣required.');
            }
            return $next($request);
        }
    }
```

Listing 3 shows the case-assignment authorisation trait, applied in `CaseController`, `EvidenceController`, `CustodyController`, and `ReportController` on every method that loads a specific case or its evidence.

Listing 3: `app/Http/Controllers/Concerns/AuthorizesCaseAccess.php`

```
    trait AuthorizesCaseAccess
    {
        protected function authorizeCaseAccess(ForensicCase $case)
            : void
        {
            $user = Auth::user();
            if (in_array($user->role, ['Administrator', '
                Reviewer'])) {
                return;
            }
            if (!$case->assignedUsers->contains($user->id)) {
                abort(403, 'Access␣denied.␣You␣are␣not␣
                    assigned␣to␣this␣case.');
            }
        }
    }
```

This trait is itself the fix for an insecure-direct-object-reference (IDOR) gap found during the self-audit in Section 5.3, Finding 5: only `CaseController` originally enforced case-assignment authorisation, meaning `EvidenceController`, `CustodyController`, and `ReportController` could be reached directly with a guessed or enumerated case ID by an authenticated user not assigned to that case.

4.3 Evidence Intake and Hashing

On upload, the server computes both a SHA-256 and an MD5 digest of the file before it is written to storage; the hash is never accepted from the client. Each evidence item is tagged with a classification (original, forensic copy, export, screenshot, or reconstructed) at intake, along with source device, collection date/time, and location, matching the fields identified in the SWGDE-informed review in Section 2.1.

4.4 Chain-of-Custody Workflow

A custody transfer is created in a `pending` state naming a sender and a receiver; it only becomes `accepted` once the receiver confirms it, and can otherwise be `rejected`. This means a transfer of responsibility cannot be logged unilaterally – both parties' actions are on record, directly implementing the ISO/IEC 27037 principle discussed in Section 2.1.

4.5 Tamper-Evident Audit Logging

Every consequential action – login, view, upload, download, transfer, export – is written to `audit_logs` through `AuditLoggerService`. Each new entry's `entry_hash` is computed from that entry's own contents together with the previous entry's `entry_hash`, chaining every row into a single sequence. A verification routine, `verifyChainIntegrity()`, walks the chain from the beginning and flags the first point at which a stored hash no longer matches what its contents recompute to.

Listing 4 shows the entry-creation logic: each new row's hash is computed over its own payload plus the previous row's hash, and the previous entry is read fresh from the database rather than cached, so the chain cannot be bypassed by stale application state.

Listing 4: `AuditLoggerService::log()`

```
public static function log(string $actionType, ?string
    $targetType = null, ?int $targetId = null, array $details =
    []): AuditLog
{
        $userId = Auth::id();
        $previousEntry = AuditLog::orderBy('id', 'desc')->first();
        $previousHash = $previousEntry ? $previousEntry->
            entry_hash
        : '00000000000000000000000000000000' . '
            00000000000000000000000000000000';
        $timestamp = now()->toIso8601String();
        $jsonDetails = json_encode($details);
        $payloadToHash = sprintf(
        '%s|%s|%s|%s|%s|%s|%s',
        $userId ?? 'system', $actionType, $targetType ?? '',
            $targetId ?? '', $jsonDetails, $timestamp,
            $previousHash
        );
        $entryHash = hash('sha256', $payloadToHash);
        return AuditLog::create([
        'user_id' => $userId, 'action_type' => $actionType, '
            target_type' => $targetType,
```

```
            'target_id' => $targetId, 'details_json' => $details,
            'entry_hash' => $entryHash, 'previous_entry_hash' =>
                $previousHash, 'created_at' => $timestamp,
            ]);
    }
```

Listing 5 shows the corresponding verification routine, which walks the chain from the oldest entry forward, recomputing each entry's expected hash and comparing it against both the stored value and the hash the following row claims as its predecessor.

Listing 5: `AuditLoggerService::verifyChainIntegrity()`

```
    public static function verifyChainIntegrity(): array
    {
        $logs = AuditLog::orderBy('id', 'asc')->get();
        $tamperedLogs = [];
        $expectedPreviousHash = '0000000000000000000000000000000000000000'
        . '0000000000000000000000000000000000000000';
        foreach ($logs as $log) {
            if ($log->previous_entry_hash !==
                $expectedPreviousHash) {
                $tamperedLogs[] = ['id' => $log->id, 'reason
                    ' => 'Previous␣entry␣hash␣mismatch'];
            }
            $payloadToHash = sprintf('%s|%s|%s|%s|%s|%s|%s',
            $log->user_id ?? 'system', $log->action_type, $log
                ->target_type ?? '', $log->target_id ?? '',
            json_encode($log->details_json), $log->created_at->
                toIso8601String(), $log->previous_entry_hash);
            if ($log->entry_hash !== hash('sha256',
                $payloadToHash)) {
                $tamperedLogs[] = ['id' => $log->id, 'reason
                    ' => 'Calculated␣payload␣hash␣mismatch'
                    ];
            }
            $expectedPreviousHash = $log->entry_hash;
        }
        return ['is_valid' => count($tamperedLogs) === 0, '
            total_entries' => count($logs), 'tampered_entries' =>
            $tamperedLogs];
    }
```

4.6 Reporting

Chain-of-custody and case-summary reports are generated through `ReportController`. During the final week of development, a defect was found and fixed here: both report endpoints were returning raw HTML with an inline `Content-Type`, rather than an actual downloadable PDF, despite being described in the design documents as court-ready documentation. The fix, detailed in Section 5.3 (Finding 1), integrated `barryvdh/laravel-dompdf` so both endpoints now generate and serve a real PDF file. As designed, each generated report embeds a manifest: a hash of everything the report contains, itself stored in the `reports` table, so the exported document can later be verified as unaltered.

4.7 Supporting Features

A batch export endpoint packages multiple evidence items into a single ZIP archive. The case detail view includes an interactive timeline of case events, and the evidence detail view includes an in-browser hex viewer and EXIF metadata display for supported file types. Case notes and a notification centre support day-to-day collaboration around a case, and the admin console exposes user management, a global evidence vault view, and system settings to Administrator-role users.

5 Results and Findings

5.1 Functional Verification

The following figures demonstrate the platform's core workflows end to end, from authentication through evidence handling to administrative oversight.

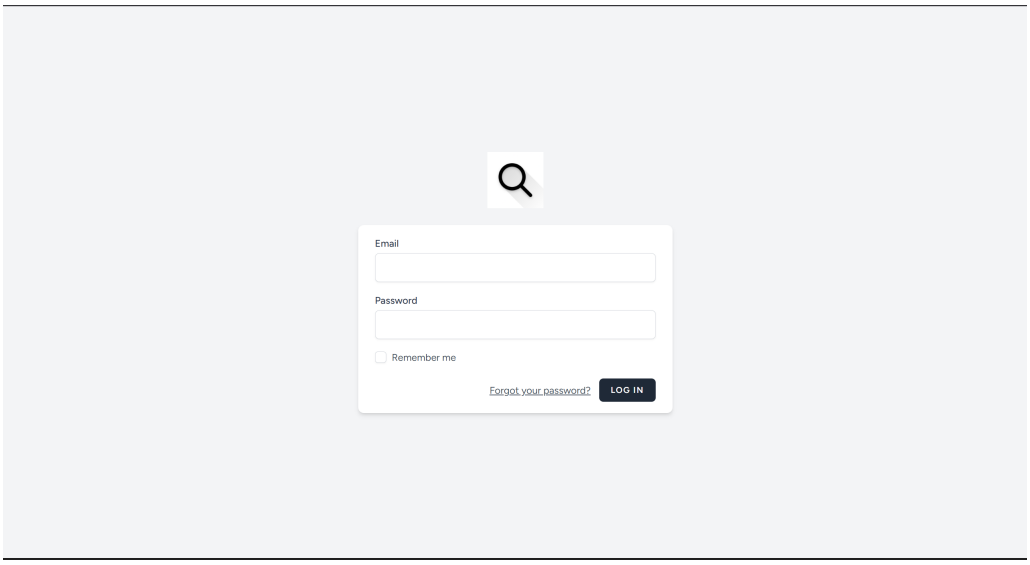

Figure 3: Investigator login screen

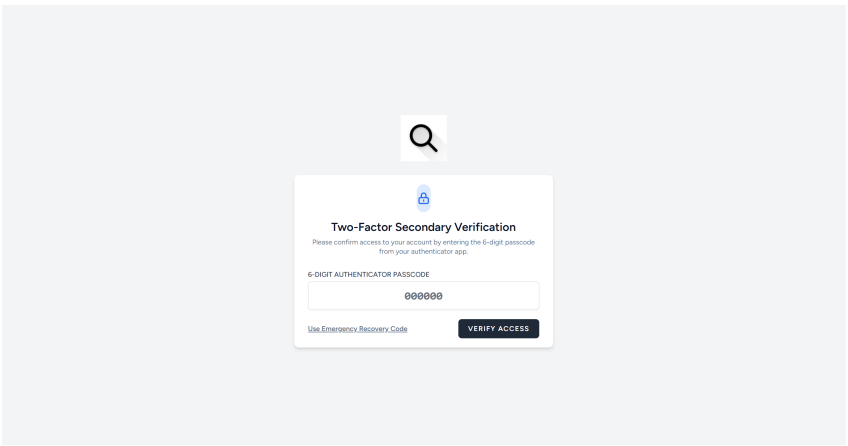

Figure 4: TOTP two-factor authentication challenge

Figures 3–14 are referenced individually in the discussion above and should each be cropped tightly to the relevant part of the screen before the report is printed, per the formatting requirement in the submission guidelines.

5.2 Tamper-Evidence Test

To demonstrate the audit log's tamper-evidence claim concretely rather than only describing it, the following test was designed and executed against the live audit log:

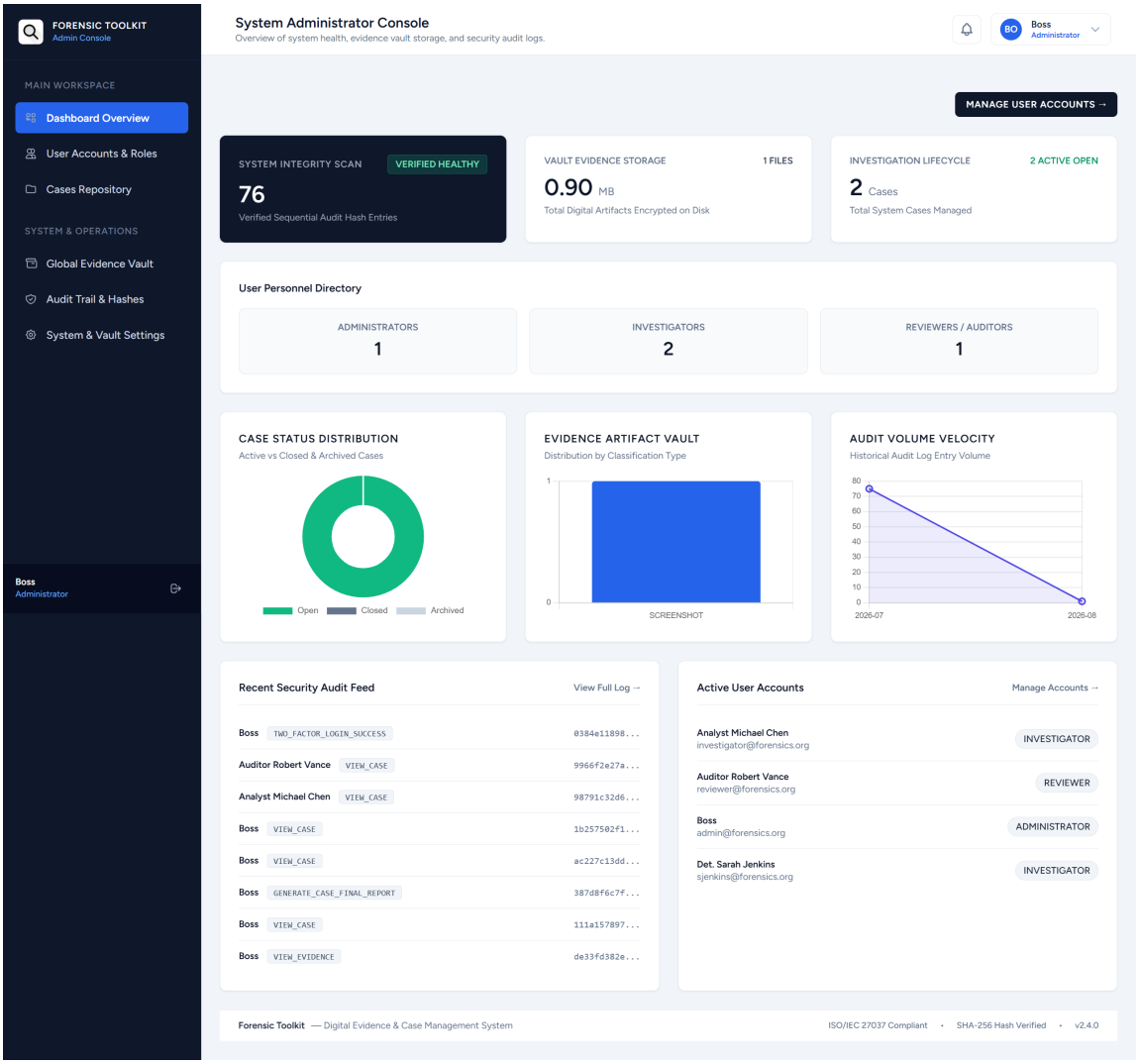

Figure 5: System Administrator Console: system integrity scan, vault storage, case status distribution, evidence classification, and audit volume analytics

1. Select an existing entry in `audit_logs` and modify one field directly at the database level, simulating an actor with direct database access bypassing the application layer entirely.
2. Trigger the integrity scan (`AuditLoggerService::verifyChainIntegrity()`, Listing 5, backing the admin audit-scan route).
3. Confirm that the scan identifies the modified row as tampered.
4. Restore the original value and re-scan, to confirm the modification left no lasting corruption.

Table 3 records the actual result.

Table 3: Tamper-evidence test result

Step	**Result**
Baseline check	`is_valid: true`, 83 entries, 0 tampered

Step	Result
Tampered log entry #7 directly (`action_type` changed to `view_case_TAMPERED_FOR_TEST`, bypassing the application entirely)	—
Re-ran integrity check	`is_valid: false`, 83 entries, 1 tampered entry detected: `{"id":7,"reason":"Calculated payload hash mismatch"}`
Restored original value	`is_valid: true`, 0 tampered – confirmed no lasting corruption

The test confirms the chain detects a directly-modified row. One precise caveat is worth stating rather than glossing over: only the single tampered row was flagged, not a cascading break through every subsequent entry. This is because `verifyChainIntegrity()` links entries using each row's own stored `entry_hash` value as the next row's expected predecessor, rather than a freshly recalculated one – so the chain reliably catches accidental or naive tampering (exactly what this test demonstrates), but is not cryptographically hardened against a sophisticated attacker who also recomputes and overwrites `entry_hash` (and every downstream `previous_entry_hash`) to match forged content, since the hash is a plain SHA-256 of visible fields with no server-only secret or external anchor involved. This distinction, and a recommended fix, is carried into Section 6.3.

5.3 Security Self-Audit

A self-audit was conducted against the live codebase on 2 August 2026, informed by the control categories in the OWASP Application Security Verification Standard [8] (authentication, access control, and error handling in particular). Table 4 lists the findings.

Table 4: Security self-audit findings

#	Finding	Severity	Status
1	Chain-of-custody and case-summary report endpoints returned raw HTML with an inline `Content-Type` instead of a real downloadable PDF, despite being described as court-ready documents	Low (integrity/ functional gap, not directly exploitable)	**Fixed** 2 Aug 2026 – `barryvdh/laravel-dompdf` integrated; both endpoints now serve a real PDF, confirmed via `artisan tinker` to emit a valid `%PDF-1.7` byte header rather than HTML

#	Finding	Severity	Status
2	`laravel/framework` 11.55.0 carries open security advisories, including a signed-URL path-confusion issue and a CRLF-injection issue in the default email validation rule used across login, registration, admin user creation, and password reset	Medium/High per advisory	**Open, risk accepted** – no patched 11.x release exists; remediation requires a major-version upgrade to Laravel 12.60+/13.10+. Given a local, non-internet-facing demo deployment for grading, real-world exploitability is low; a major-version upgrade under submission deadline pressure was judged riskier than the residual risk itself. Documented here as a deliberate, reasoned decision rather than an oversight.
3	`User::$fillable` includes `role`, raising the question of whether any controller mass-assigns from unfiltered request input, which would let a user set their own role	Needs verification	**Partially investigated** – `RegisteredUserController` and `ProfileController::update` were checked and do not expose this path (they build the update array explicitly / use a validated Form Request). A full sweep of admin user-creation and case-team-assignment controllers was not completed; flagged as follow-up work.
4	`APP_DEBUG=true` in `.env`	Informational (expected in local development)	Acceptable for the local grading demo; flagged as a hard requirement to set to `false` before any real deployment, since a true value leaks file paths, environment values, and database structure through unhandled error pages.
5	Insecure direct object reference (IDOR): case-assignment authorisation was originally enforced only in `CaseController`, meaning `EvidenceController`, `CustodyController`, and `Report Controller` could be reached with a guessed or enumerated case ID by an authenticated user not assigned to that case	High (authenticated users could view or act on other investigators' case evidence)	**Fixed** 2 Aug 2026 – introduced the `AuthorizesCaseAccess` trait (Section 4.2, Listing 3) and applied it across all four controllers on every method that loads a specific case or its evidence

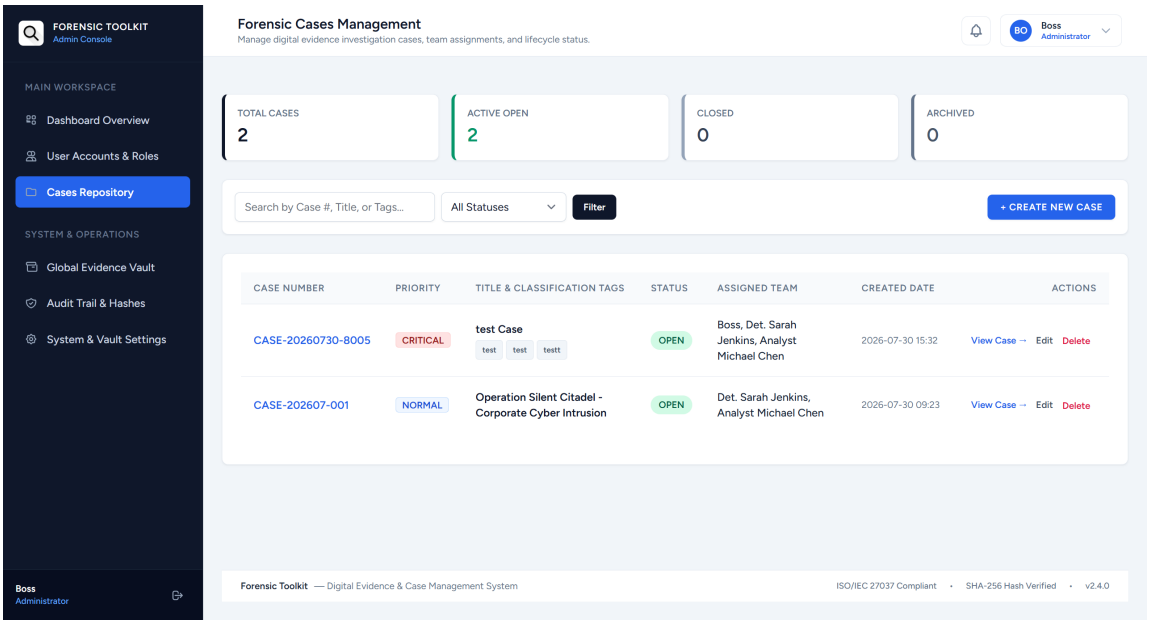

Figure 6: Forensic Cases Management: case repository listing with status and team assignment

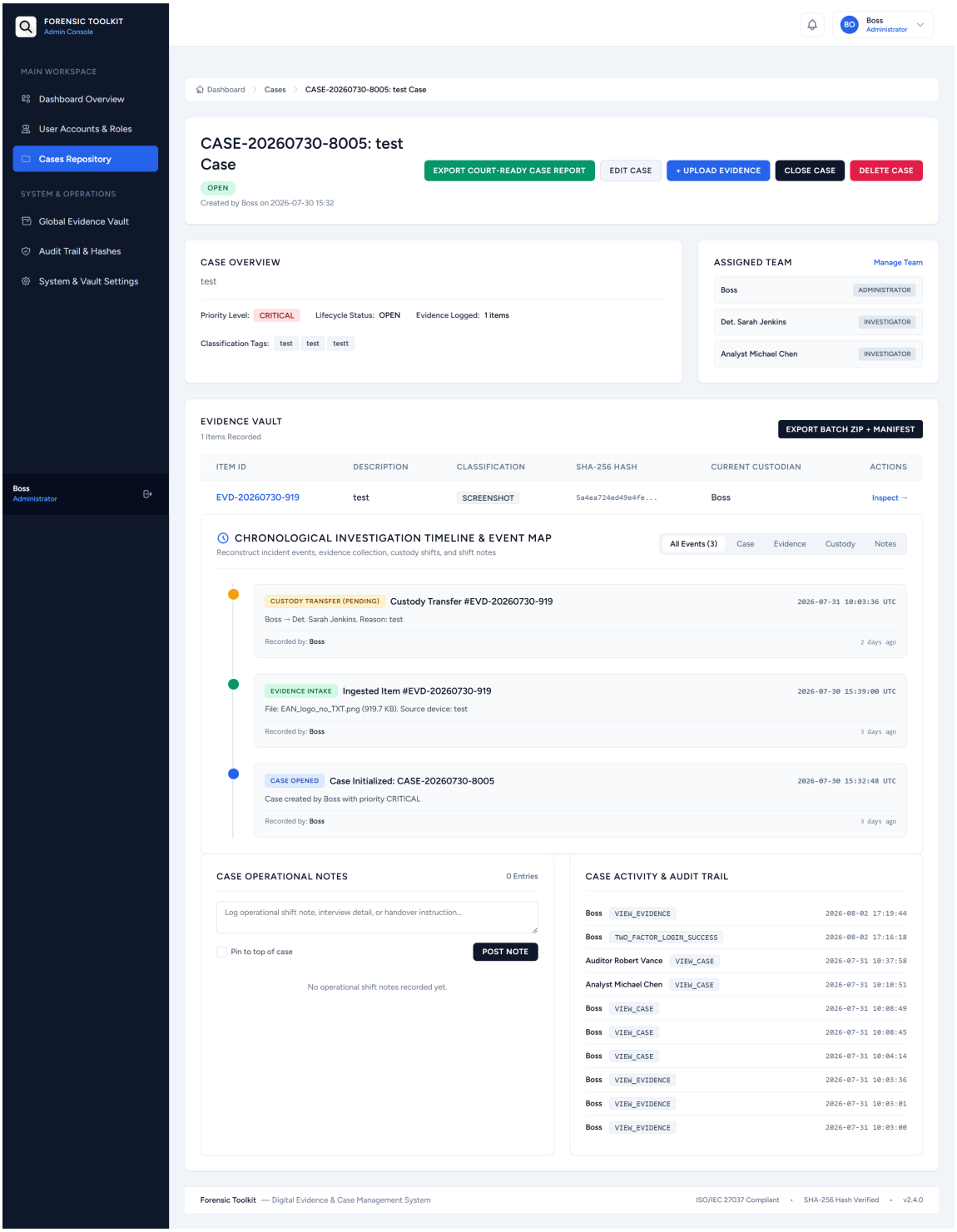

Figure 7: Individual case view, including assigned team, evidence vault, and chronological investigation timeline

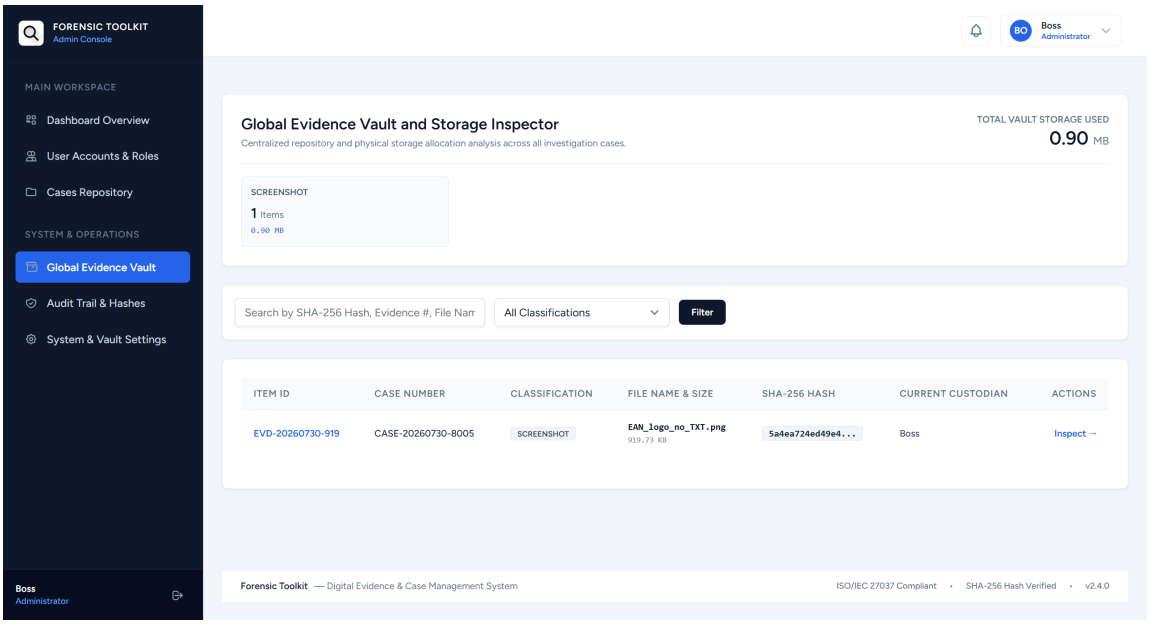

Figure 8: Global Evidence Vault and storage inspector

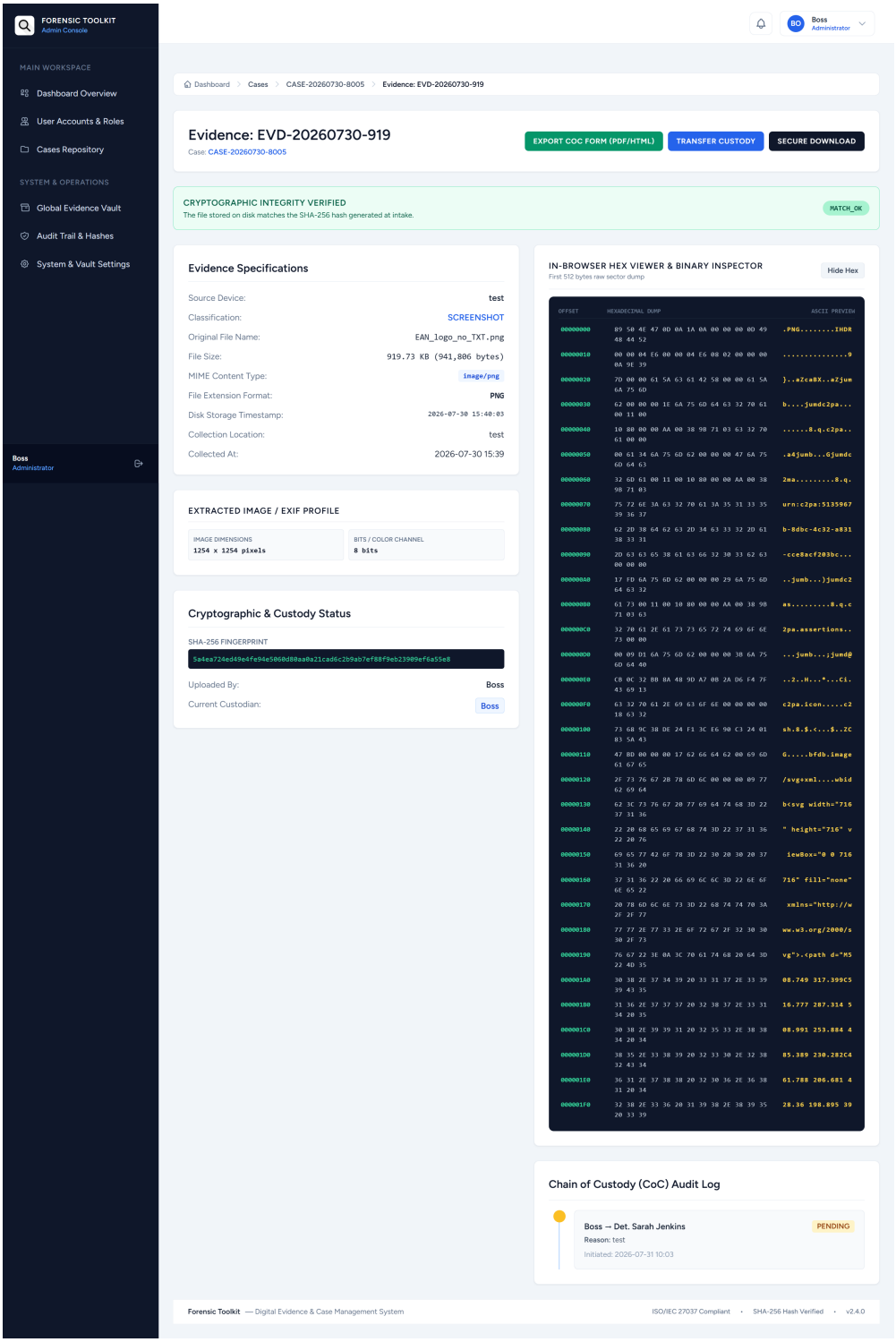

Figure 9: Evidence detail view: cryptographic integrity status, EXIF profile, in-browser hex viewer, and custody status

Figure 10: Audit Trail & Hashes: tamper-evident, hash-chained audit log with per-entry linked hashes

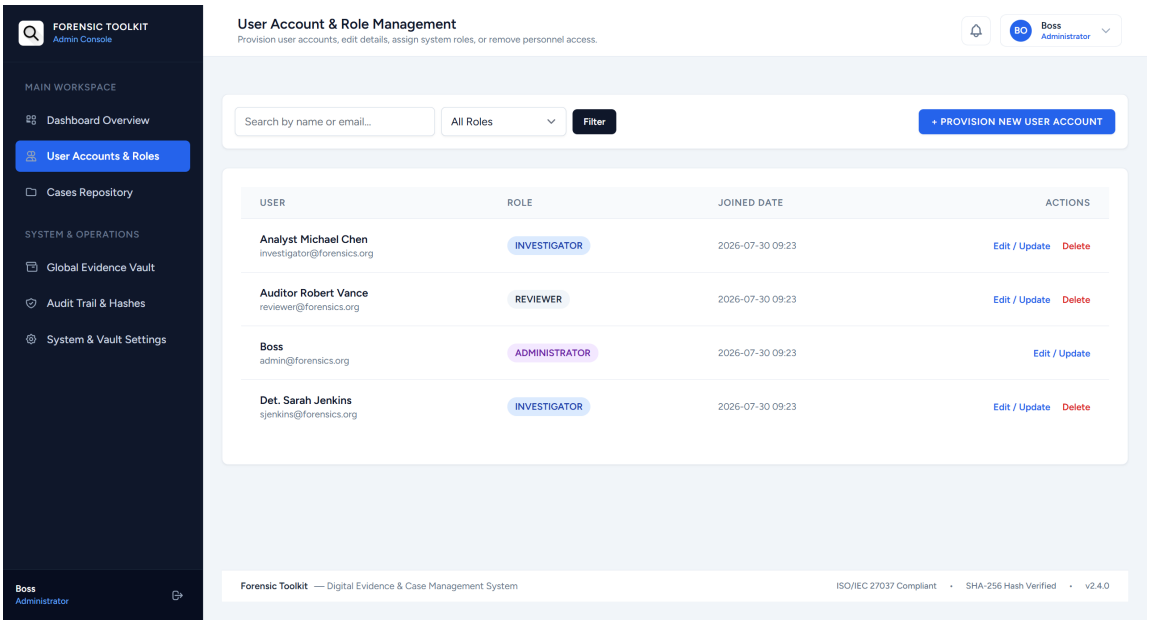

Figure 11: User Account & Role Management console (role-based access control)

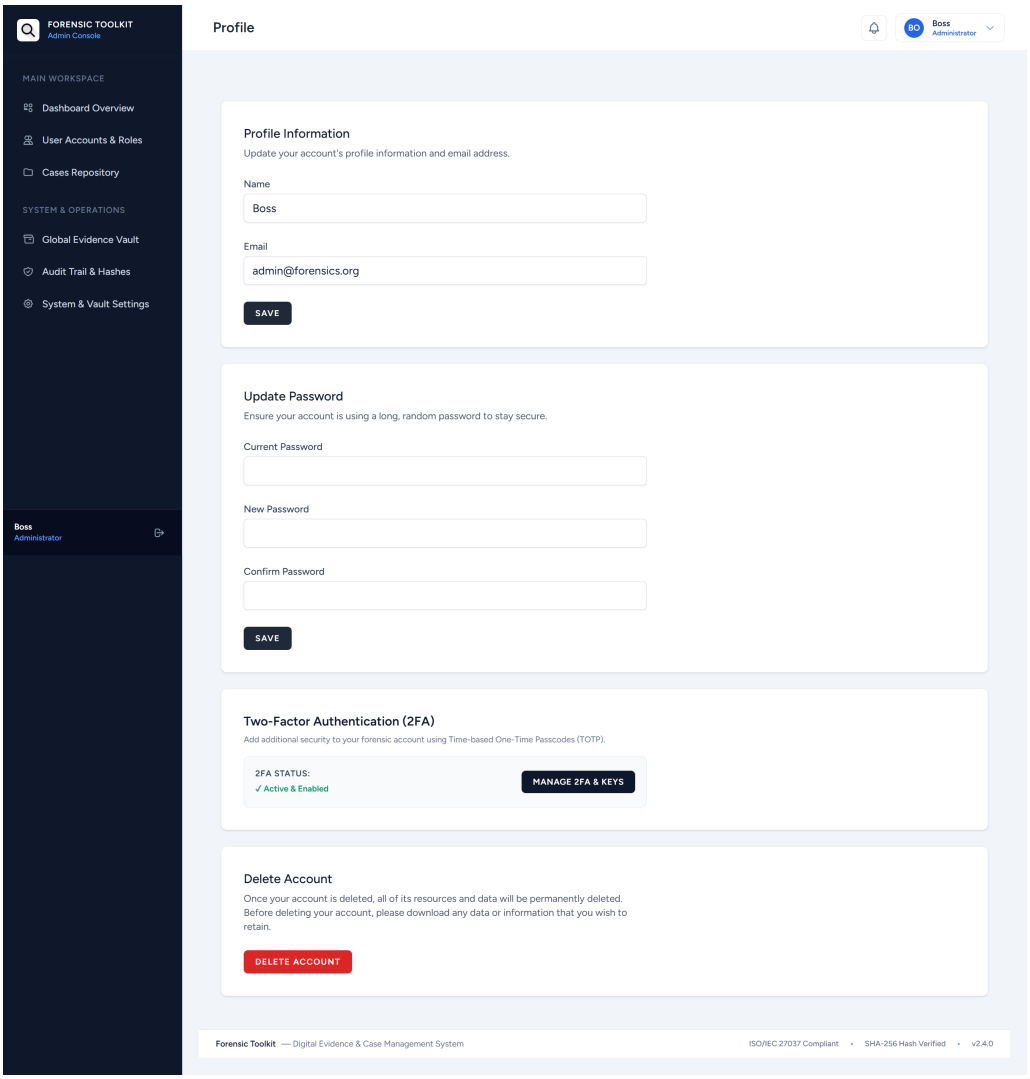

Figure 12: User profile, password, and two-factor account settings

FORENSIC TOOLKIT - OFFICIAL CASE REPORT

DIGITAL EVIDENCE & FORENSIC CASE MANAGEMENT SYSTEM

CLASSIFIED / COURT READY

Case Number: CASE-20260730-8005
Title: test Case
Priority Level: CRITICAL
Classification Tags: test,test,testt

Lifecycle Status: OPEN
Lead Investigator / Creator: Boss
Created Date: 2026-07-30 15:32:48 UTC
Report Generated: 2026-08-02T17:32:43+00:00

I. EXECUTIVE SUMMARY & OVERVIEW

test

II. ASSIGNED INVESTIGATION TEAM

PERSONNEL NAME	ROLE / DESIGNATION	EMAIL ADDRESS
Boss	Administrator	admin@forensics.org
Det. Sarah Jenkins	Investigator	sjenkins@forensics.org
Analyst Michael Chen	Investigator	investigator@forensics.org

Figure 13: Generated Case Final Report, downloaded as a real PDF (page 1 of 2) – confirms the Section 4.6 / Table 5 Finding 1 fix

III. INGESTED EVIDENCE VAULT ITEMS

EVIDENCE ID	SOURCE DEVICE	CLASSIFICATION	SHA-256 FINGERPRINT	CURRENT CUSTODIAN
EVD-20260730-919	test	screenshot	5a4ea724ed49e4fe94e5060d80aa0a21cad6c2b9ab7ef88f9eb23909ef6a55e8	Boss

IV. COMPLETE CASE ACTIVITY & AUDIT HISTORY

TIMESTAMP	USER / ACTOR	ACTION TYPE	TARGET DETAILS
2026-08-02 17:32:39	Boss	view_case	null
2026-08-02 17:21:39	Boss	view_case	null
2026-08-02 17:20:12	Boss	view_case	null
2026-08-02 17:19:44	Boss	view_evidence	null
2026-08-02 17:16:18	Boss	two_factor_login_success	null
2026-07-31 10:37:58	Auditor Robert Vance	view_case	null
2026-07-31 10:10:51	Analyst Michael Chen	view_case	null

Figure 14: Generated Case Final Report PDF (page 2 of 2): evidence vault items and complete case activity/audit history

6 Analysis and Recommendations

6.1 Design-versus-Implementation Gap Analysis

Table 5 compares what the original design documents (the PRD and TRD) specified against what is actually implemented today. Documenting this gap honestly is more useful, and more defensible in the interview, than presenting the design documents' intentions as if they were already built.

Table 5: Design intent versus current implementation

Design requirement	**Current status**	**Gap**
Evidence files encrypted at rest (AES-256)	Stored on Laravel's local filesystem disk, no application-level encryption	Not implemented
Virus/malware scanning on upload	Not implemented	Not implemented
HTTPS/TLS + Cloudflare WAF	Not applicable to local development environment	Deployment-stage requirement, not yet exercised
Chunked/resumable uploads for large forensic images	Single-request upload only	Not implemented
SHA-256 hashing on upload	Implemented, server-side	None
Chain-of-custody with explicit acceptance	Implemented	None
Hash-chained tamper-evident audit log	Implemented, with verification routine	None
Role-based access control	Implemented at middleware/controller level	None (full self-escalation sweep still pending, see Table 4, Finding 3)
Court-ready PDF export with integrity manifest	Implemented (fixed 2 Aug 2026)	None

6.2 Risk Acceptance on Framework Vulnerabilities

The decision to leave the Laravel framework's open advisories unresolved (Table 4, Finding 2) rather than attempt a major-version upgrade in the final week reflects a considered trade-off rather than an oversight: an untested major-version migration this close to a submission deadline risks breaking working functionality across authentication, evidence handling, and reporting simultaneously, for a vulnerability class whose real-world exploitability is low in a local, non-internet-facing demo context. In a production deployment this calculus would reverse – the upgrade would need to happen before go-live, not after.

6.3 Prioritised Recommendations

1. **At-rest encryption for evidence files.** Introduce an encryption layer (e.g. Laravel's `Crypt` facade around file contents, or storage-level AES-256) before any file is written to disk. This directly closes the largest gap against ISO/IEC 27037's preservation guidance.

2. **Harden the audit log against a full-database-access attacker.** As identified in Section 5.2's tamper-evidence test, `entry_hash` is currently a plain SHA-256 of visible fields with no server-only secret, so an attacker able to rewrite an entire row (and every downstream `previous_entry_hash` to match) could in principle produce a chain that re-validates. Sign each entry with a server-held HMAC key that is never derivable from the visible fields, so recomputing the chain requires the key, not just the data.

3. **Plan and test a Laravel major-version upgrade** to resolve the open framework advisories, on a schedule that allows for regression testing rather than under deadline pressure.

4. **Malware scanning on upload**, quarantining rather than rejecting flagged files, since the file itself may still need to be retained as evidence.

5. **Chunked/resumable uploads** to support the multi-gigabyte forensic images the platform is ultimately meant to handle.

6. **Complete the RBAC mass-assignment sweep** across all controllers that create or update user roles, to close out Finding 3 with certainty rather than partial verification.

7. **Set** `APP_DEBUG=false` as a mandatory pre-deployment checklist item.

7 Conclusion

This project set out to replace an error-prone, paper-and-spreadsheet approach to digital evidence handling with a system that can prove, rather than merely assert, that evidence has not been tampered with. The resulting platform implements the core requirements identified in the design phase: server-side hashing on intake, an explicit two-party custody-transfer workflow, a hash-chained and independently verifiable audit log, role- and case-based access control, two-factor authentication, and PDF reporting with an embedded integrity manifest.

Equally important to the system itself is the process that produced this report: a genuine self-audit of the completed codebase, conducted in the same spirit as the auditing practices this course teaches, which surfaced and fixed a real defect in report generation and produced an honest accounting of what the original design called for versus what currently exists. The gaps that remain – encryption at rest, malware scanning, and the framework's open advisories chief among them – are documented rather than concealed, along with a reasoned argument for why each was deprioritised for this phase. A system that can account for its own shortcomings this precisely is, in a small way, practising the exact principle it was built to support: that trust in a record should rest on evidence, not on assurance.

References

References

[1] K. Kent, S. Chevalier, T. Grance, and H. Dang, *Guide to Integrating Forensic Techniques into Incident Response*, NIST Special Publication 800-86, National Institute of Standards and Technology, Aug. 2006.

[2] International Organization for Standardization, *ISO/IEC 27037:2012 – Information technology – Security techniques – Guidelines for identification, collection, acquisition, and preservation of digital evidence*, ISO/IEC, 2012.

[3] Scientific Working Group on Digital Evidence, *SWGDE Best Practices for Digital Evidence Collection*, Document 18-F-002, Version 2.0, SWGDE, Nov. 2025.

[4] B. Schneier and J. Kelsey, "Secure Audit Logs to Support Computer Forensics," *ACM Transactions on Information and System Security*, vol. 2, no. 2, pp. 159–176, May 1999.

[5] InterNational Committee for Information Technology Standards, *INCITS 359-2012 (R2022) – Information Technology – Role Based Access Control*, ANSI/INCITS, 2012 (reaffirmed 2022).

[6] Laravel LLC, *Laravel 11.x Documentation*, https://laravel.com/docs/11.x, accessed Aug. 2026.

[7] B. van Dooren, *laravel-dompdf: A DOMPDF Wrapper for Laravel*, GitHub repository, https://github.com/barryvdh/laravel-dompdf, accessed Aug. 2026.

[8] OWASP Foundation, *OWASP Application Security Verification Standard (ASVS)*, https://owasp.org/www-project-application-security-verification-standard/, accessed Aug. 2026.

Appendices

Appendix A: Full Database Schema

Exact `SHOW CREATE TABLE` output for every table in the live database, captured 2 August 2026. All personnel data shown ("Boss", "Det. Sarah Jenkins", `@forensics.org` addresses) is fake lab data created for grading purposes, not real personal data, so no redaction is required under the submission guidelines' redaction rule; this should be reconfirmed for any data added after this capture.

```
CREATE TABLE `users` (
`id` bigint(20) unsigned NOT NULL AUTO_INCREMENT,
`name` varchar(255) NOT NULL,
`email` varchar(255) NOT NULL,
`email_verified_at` timestamp NULL DEFAULT NULL,
`password` varchar(255) NOT NULL,
`role` varchar(255) NOT NULL DEFAULT 'Investigator',
`two_factor_secret` varchar(255) DEFAULT NULL,
`two_factor_recovery_codes` text DEFAULT NULL,
`two_factor_confirmed_at` timestamp NULL DEFAULT NULL,
`remember_token` varchar(100) DEFAULT NULL,
`created_at` timestamp NULL DEFAULT NULL,
`updated_at` timestamp NULL DEFAULT NULL,
PRIMARY KEY (`id`),
UNIQUE KEY `users_email_unique` (`email`)
) ENGINE=InnoDB AUTO_INCREMENT=6 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

CREATE TABLE `password_reset_tokens` (
`email` varchar(255) NOT NULL,
`token` varchar(255) NOT NULL,
`created_at` timestamp NULL DEFAULT NULL,
PRIMARY KEY (`email`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

CREATE TABLE `sessions` (
`id` varchar(255) NOT NULL,
`user_id` bigint(20) unsigned DEFAULT NULL,
`ip_address` varchar(45) DEFAULT NULL,
`user_agent` text DEFAULT NULL,
`payload` longtext NOT NULL,
`last_activity` int(11) NOT NULL,
PRIMARY KEY (`id`),
KEY `sessions_user_id_index` (`user_id`),
KEY `sessions_last_activity_index` (`last_activity`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

CREATE TABLE `cache` (
`key` varchar(255) NOT NULL,
`value` mediumtext NOT NULL,
`expiration` int(11) NOT NULL,
PRIMARY KEY (`key`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

CREATE TABLE `cache_locks` (
`key` varchar(255) NOT NULL,
`owner` varchar(255) NOT NULL,
```

```
`expiration` int(11) NOT NULL,
PRIMARY KEY (`key`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

CREATE TABLE `jobs` (
`id` bigint(20) unsigned NOT NULL AUTO_INCREMENT,
`queue` varchar(255) NOT NULL,
`payload` longtext NOT NULL,
`attempts` tinyint(3) unsigned NOT NULL,
`reserved_at` int(10) unsigned DEFAULT NULL,
`available_at` int(10) unsigned NOT NULL,
`created_at` int(10) unsigned NOT NULL,
PRIMARY KEY (`id`),
KEY `jobs_queue_index` (`queue`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

CREATE TABLE `job_batches` (
`id` varchar(255) NOT NULL,
`name` varchar(255) NOT NULL,
`total_jobs` int(11) NOT NULL,
`pending_jobs` int(11) NOT NULL,
`failed_jobs` int(11) NOT NULL,
`failed_job_ids` longtext NOT NULL,
`options` mediumtext DEFAULT NULL,
`cancelled_at` int(11) DEFAULT NULL,
`created_at` int(11) NOT NULL,
`finished_at` int(11) DEFAULT NULL,
PRIMARY KEY (`id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

CREATE TABLE `failed_jobs` (
`id` bigint(20) unsigned NOT NULL AUTO_INCREMENT,
`uuid` varchar(255) NOT NULL,
`connection` text NOT NULL,
`queue` text NOT NULL,
`payload` longtext NOT NULL,
`exception` longtext NOT NULL,
`failed_at` timestamp NOT NULL DEFAULT current_timestamp(),
PRIMARY KEY (`id`),
UNIQUE KEY `failed_jobs_uuid_unique` (`uuid`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

CREATE TABLE `cases` (
`id` bigint(20) unsigned NOT NULL AUTO_INCREMENT,
`case_number` varchar(255) NOT NULL,
`title` varchar(255) NOT NULL,
`description` text DEFAULT NULL,
`status` enum('open','closed','archived') NOT NULL DEFAULT 'open',
`priority` varchar(255) NOT NULL DEFAULT 'Normal',
`tags` varchar(255) DEFAULT NULL,
`created_by` bigint(20) unsigned NOT NULL,
`closed_at` timestamp NULL DEFAULT NULL,
`created_at` timestamp NULL DEFAULT NULL,
`updated_at` timestamp NULL DEFAULT NULL,
`deleted_at` timestamp NULL DEFAULT NULL,
PRIMARY KEY (`id`),
UNIQUE KEY `cases_case_number_unique` (`case_number`),
KEY `cases_created_by_foreign` (`created_by`),
CONSTRAINT `cases_created_by_foreign` FOREIGN KEY (`created_by`) REFERENCES `users` (`id`)
```

```
    ON DELETE CASCADE
) ENGINE=InnoDB AUTO_INCREMENT=5 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

CREATE TABLE `case_assignments` (
`id` bigint(20) unsigned NOT NULL AUTO_INCREMENT,
`case_id` bigint(20) unsigned NOT NULL,
`user_id` bigint(20) unsigned NOT NULL,
`role_in_case` varchar(255) NOT NULL DEFAULT 'Investigator',
`assigned_at` timestamp NOT NULL DEFAULT current_timestamp(),
`created_at` timestamp NULL DEFAULT NULL,
`updated_at` timestamp NULL DEFAULT NULL,
PRIMARY KEY (`id`),
UNIQUE KEY `case_assignments_case_id_user_id_unique` (`case_id`,`user_id`),
KEY `case_assignments_user_id_foreign` (`user_id`),
CONSTRAINT `case_assignments_case_id_foreign` FOREIGN KEY (`case_id`) REFERENCES `cases` (`
    id`) ON DELETE CASCADE,
CONSTRAINT `case_assignments_user_id_foreign` FOREIGN KEY (`user_id`) REFERENCES `users` (`
    id`) ON DELETE CASCADE
) ENGINE=InnoDB AUTO_INCREMENT=6 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

CREATE TABLE `evidence_items` (
`id` bigint(20) unsigned NOT NULL AUTO_INCREMENT,
`case_id` bigint(20) unsigned NOT NULL,
`parent_id` bigint(20) unsigned DEFAULT NULL,
`evidence_number` varchar(255) NOT NULL,
`description` text NOT NULL,
`source_device` varchar(255) NOT NULL,
`classification` enum('original','forensic_copy','export','screenshot','reconstructed') NOT
    NULL DEFAULT 'original',
`custom_classification` varchar(255) DEFAULT NULL,
`file_path` varchar(255) NOT NULL,
`file_name` varchar(255) NOT NULL,
`file_hash_sha256` varchar(255) NOT NULL,
`file_size` bigint(20) unsigned NOT NULL,
`uploaded_by` bigint(20) unsigned NOT NULL,
`collected_at` timestamp NOT NULL DEFAULT current_timestamp() ON UPDATE current_timestamp(),
`collected_location` varchar(255) NOT NULL,
`current_custodian_id` bigint(20) unsigned NOT NULL,
`created_at` timestamp NULL DEFAULT NULL,
`updated_at` timestamp NULL DEFAULT NULL,
PRIMARY KEY (`id`),
UNIQUE KEY `evidence_items_evidence_number_unique` (`evidence_number`),
KEY `evidence_items_case_id_foreign` (`case_id`),
KEY `evidence_items_uploaded_by_foreign` (`uploaded_by`),
KEY `evidence_items_current_custodian_id_foreign` (`current_custodian_id`),
KEY `evidence_items_parent_id_foreign` (`parent_id`),
CONSTRAINT `evidence_items_case_id_foreign` FOREIGN KEY (`case_id`) REFERENCES `cases` (`id
    `) ON DELETE CASCADE,
CONSTRAINT `evidence_items_current_custodian_id_foreign` FOREIGN KEY (`current_custodian_id
    `) REFERENCES `users` (`id`),
CONSTRAINT `evidence_items_parent_id_foreign` FOREIGN KEY (`parent_id`) REFERENCES `
    evidence_items` (`id`) ON DELETE CASCADE,
CONSTRAINT `evidence_items_uploaded_by_foreign` FOREIGN KEY (`uploaded_by`) REFERENCES `
    users` (`id`)
) ENGINE=InnoDB AUTO_INCREMENT=3 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

CREATE TABLE `custody_transfers` (
`id` bigint(20) unsigned NOT NULL AUTO_INCREMENT,
`evidence_item_id` bigint(20) unsigned NOT NULL,
```

```
`from_user_id` bigint(20) unsigned NOT NULL,
`to_user_id` bigint(20) unsigned NOT NULL,
`reason` text NOT NULL,
`transferred_at` timestamp NOT NULL DEFAULT current_timestamp(),
`accepted_at` timestamp NULL DEFAULT NULL,
`status` enum('pending','accepted','rejected') NOT NULL DEFAULT 'pending',
`created_at` timestamp NULL DEFAULT NULL,
`updated_at` timestamp NULL DEFAULT NULL,
PRIMARY KEY (`id`),
KEY `custody_transfers_evidence_item_id_foreign` (`evidence_item_id`),
KEY `custody_transfers_from_user_id_foreign` (`from_user_id`),
KEY `custody_transfers_to_user_id_foreign` (`to_user_id`),
CONSTRAINT `custody_transfers_evidence_item_id_foreign` FOREIGN KEY (`evidence_item_id`)
    REFERENCES `evidence_items` (`id`) ON DELETE CASCADE,
CONSTRAINT `custody_transfers_from_user_id_foreign` FOREIGN KEY (`from_user_id`) REFERENCES
    `users` (`id`),
CONSTRAINT `custody_transfers_to_user_id_foreign` FOREIGN KEY (`to_user_id`) REFERENCES `
    users` (`id`)
) ENGINE=InnoDB AUTO_INCREMENT=2 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

CREATE TABLE `audit_logs` (
`id` bigint(20) unsigned NOT NULL AUTO_INCREMENT,
`user_id` bigint(20) unsigned DEFAULT NULL,
`action_type` varchar(255) NOT NULL,
`target_type` varchar(255) DEFAULT NULL,
`target_id` bigint(20) unsigned DEFAULT NULL,
`details_json` longtext CHARACTER SET utf8mb4 COLLATE utf8mb4_bin DEFAULT NULL CHECK (
    json_valid(`details_json`)),
`entry_hash` varchar(255) NOT NULL,
`previous_entry_hash` varchar(255) DEFAULT NULL,
`created_at` timestamp NOT NULL DEFAULT current_timestamp(),
PRIMARY KEY (`id`),
KEY `audit_logs_user_id_foreign` (`user_id`),
KEY `audit_logs_target_type_target_id_index` (`target_type`,`target_id`),
KEY `audit_logs_action_type_index` (`action_type`),
KEY `audit_logs_created_at_index` (`created_at`),
CONSTRAINT `audit_logs_user_id_foreign` FOREIGN KEY (`user_id`) REFERENCES `users` (`id`) ON
     DELETE SET NULL
) ENGINE=InnoDB AUTO_INCREMENT=85 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

CREATE TABLE `reports` (
`id` bigint(20) unsigned NOT NULL AUTO_INCREMENT,
`case_id` bigint(20) unsigned NOT NULL,
`evidence_item_id` bigint(20) unsigned DEFAULT NULL,
`type` varchar(255) NOT NULL,
`file_path` varchar(255) NOT NULL,
`generated_by` bigint(20) unsigned NOT NULL,
`generated_at` timestamp NOT NULL DEFAULT current_timestamp(),
`manifest_hash` varchar(255) NOT NULL,
`created_at` timestamp NULL DEFAULT NULL,
`updated_at` timestamp NULL DEFAULT NULL,
PRIMARY KEY (`id`),
KEY `reports_case_id_foreign` (`case_id`),
KEY `reports_evidence_item_id_foreign` (`evidence_item_id`),
KEY `reports_generated_by_foreign` (`generated_by`),
CONSTRAINT `reports_case_id_foreign` FOREIGN KEY (`case_id`) REFERENCES `cases` (`id`) ON
    DELETE CASCADE,
CONSTRAINT `reports_evidence_item_id_foreign` FOREIGN KEY (`evidence_item_id`) REFERENCES `
    evidence_items` (`id`) ON DELETE CASCADE,
```

```
CONSTRAINT `reports_generated_by_foreign` FOREIGN KEY (`generated_by`) REFERENCES `users` (`
    id`)
) ENGINE=InnoDB AUTO_INCREMENT=9 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

CREATE TABLE `system_settings` (
`id` bigint(20) unsigned NOT NULL AUTO_INCREMENT,
`key` varchar(255) NOT NULL,
`value` text DEFAULT NULL,
`description` varchar(255) DEFAULT NULL,
`created_at` timestamp NULL DEFAULT NULL,
`updated_at` timestamp NULL DEFAULT NULL,
PRIMARY KEY (`id`),
UNIQUE KEY `system_settings_key_unique` (`key`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

CREATE TABLE `case_notes` (
`id` bigint(20) unsigned NOT NULL AUTO_INCREMENT,
`case_id` bigint(20) unsigned NOT NULL,
`user_id` bigint(20) unsigned NOT NULL,
`note` text NOT NULL,
`is_pinned` tinyint(1) NOT NULL DEFAULT 0,
`created_at` timestamp NULL DEFAULT NULL,
`updated_at` timestamp NULL DEFAULT NULL,
PRIMARY KEY (`id`),
KEY `case_notes_case_id_foreign` (`case_id`),
KEY `case_notes_user_id_foreign` (`user_id`),
CONSTRAINT `case_notes_case_id_foreign` FOREIGN KEY (`case_id`) REFERENCES `cases` (`id`) ON
    DELETE CASCADE,
CONSTRAINT `case_notes_user_id_foreign` FOREIGN KEY (`user_id`) REFERENCES `users` (`id`) ON
    DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

CREATE TABLE `user_notifications` (
`id` bigint(20) unsigned NOT NULL AUTO_INCREMENT,
`user_id` bigint(20) unsigned NOT NULL,
`type` varchar(255) NOT NULL,
`title` varchar(255) NOT NULL,
`message` text NOT NULL,
`target_url` varchar(255) DEFAULT NULL,
`read_at` timestamp NULL DEFAULT NULL,
`created_at` timestamp NULL DEFAULT NULL,
`updated_at` timestamp NULL DEFAULT NULL,
PRIMARY KEY (`id`),
KEY `user_notifications_user_id_read_at_index` (`user_id`,`read_at`),
CONSTRAINT `user_notifications_user_id_foreign` FOREIGN KEY (`user_id`) REFERENCES `users`
    (`id`) ON DELETE CASCADE
) ENGINE=InnoDB AUTO_INCREMENT=2 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

Appendix B: Full Route List

Output of `php artisan route:list -except-vendor` (59 application routes; framework and vendor-package routes excluded as noise).

```
GET|HEAD /
GET|HEAD admin ..... admin.dashboard ->
    AdminController@dashboard
GET|HEAD admin/audit/export-csv ..... admin.audit.export-csv ->
    AuditController@exportCsv
POST admin/audit/scan ..... admin.audit.scan ->
    AuditController@runSystemScan
GET|HEAD admin/evidence ..... admin.evidence.index ->
    AdminController@globalEvidenceVault
GET|HEAD admin/settings ..... admin.settings.index ->
    AdminController@settings
POST admin/settings ..... admin.settings.update ->
    AdminController@updateSettings
GET|HEAD admin/users ..... admin.users.index ->
    AdminController@users
POST admin/users ..... admin.users.store ->
    AdminController@storeUser
PUT admin/users/{id} ..... admin.users.update ->
    AdminController@updateUser
DELETE admin/users/{id} ..... admin.users.delete ->
    AdminController@deleteUser
GET|HEAD audit ..... audit.index ->
    AuditController@index
GET|HEAD cases ..... cases.index ->
    CaseController@index
POST cases ..... cases.store ->
    CaseController@store
GET|HEAD cases/create ..... cases.create ->
     CaseController@create
POST cases/{caseId}/evidence ..... evidence.store ->
    EvidenceController@store
GET|HEAD cases/{caseId}/evidence/create ..... evidence.create ->
    EvidenceController@create
GET|HEAD cases/{caseId}/export-batch ..... evidence.batch-export ->
    EvidenceController@exportBatchZip
GET|HEAD cases/{id} ..... cases.show
    -> CaseController@show
PUT cases/{id} ..... cases.update ->
    CaseController@update
DELETE cases/{id} ..... cases.destroy ->
    CaseController@destroy
POST cases/{id}/close ..... cases.close ->
    CaseController@close
GET|HEAD cases/{id}/edit ..... cases.edit
    -> CaseController@edit
POST cases/{id}/notes ..... cases.notes.store ->
    CaseController@storeNote
POST cases/{id}/restore ..... cases.restore ->
    CaseController@restore
POST cases/{id}/status ..... cases.update-status ->
    CaseController@updateStatus
POST cases/{id}/team ..... cases.update-team ->
    CaseController@updateTeam
GET|HEAD confirm-password ..... password.confirm -> Auth\
```

```
    ConfirmablePasswordController@show
POST confirm-password ..... Auth\
    ConfirmablePasswordController@store
POST custody/{transferId}/accept ..... custody.accept ->
    CustodyController@accept
POST custody/{transferId}/reject ..... custody.reject ->
    CustodyController@reject
GET|HEAD dashboard ..... dashboard ->
    CaseController@index
POST email/verification-notification . verification.send -> Auth\
    EmailVerificationNotificationController@store
GET|HEAD evidence/{evidenceId}/transfer ..... custody.create ->
    CustodyController@create
POST evidence/{evidenceId}/transfer ..... custody.store ->
    CustodyController@store
GET|HEAD evidence/{id} ..... evidence.show ->
    EvidenceController@show
GET|HEAD evidence/{id}/download ..... evidence.download ->
    EvidenceController@download
GET|HEAD forgot-password ..... password.request -> Auth\
    PasswordResetLinkController@create
POST forgot-password ..... password.email -> Auth\
    PasswordResetLinkController@store
GET|HEAD login ..... login -> Auth\
    AuthenticatedSessionController@create
POST login ..... Auth\
    AuthenticatedSessionController@store
POST logout ..... logout -> Auth\
    AuthenticatedSessionController@destroy
PUT password ..... password.update -> Auth\
    PasswordController@update
GET|HEAD profile ..... profile.edit ->
    ProfileController@edit
PATCH profile ..... profile.update ->
    ProfileController@update
DELETE profile ..... profile.destroy ->
    ProfileController@destroy
GET|HEAD register ..... register -> Auth\
    RegisteredUserController@create
POST register ..... Auth\
    RegisteredUserController@store
GET|HEAD reports/case/{caseId} ..... reports.case ->
    ReportController@generateCaseFinalReport
GET|HEAD reports/coc/{evidenceId} ..... reports.coc ->
    ReportController@generateCoCReport
POST reset-password ..... password.store -> Auth\
    NewPasswordController@store
GET|HEAD reset-password/{token} ..... password.reset -> Auth\
    NewPasswordController@create
GET|HEAD two-factor-challenge ..... two-factor.challenge ->
    TwoFactorController@showChallenge
POST two-factor-challenge ..... two-factor.verify ->
    TwoFactorController@verifyChallenge
POST user/two-factor-disable ..... two-factor.disable ->
    TwoFactorController@disable
POST user/two-factor-enable ..... two-factor.enable ->
    TwoFactorController@enable
GET|HEAD user/two-factor-setup ..... two-factor.setup ->
    TwoFactorController@showSetup
```

```
GET|HEAD verify-email ........................... verification.notice -> Auth\
    EmailVerificationPromptController
GET|HEAD verify-email/{id}/{hash} ............................ verification.verify -> Auth\
    VerifyEmailController

Showing 59 routes
```

Appendix C: Sample Audit Log Export

A 30-row sample of the most recent entries from `admin/audit/export-csv`, showing the hash chain in action: each row's *Previous Entry Hash* matches the *Entry Hash* of the row directly above it, and any break in that sequence is exactly what `AuditLoggerService::verifyChainIntegrity()` is designed to detect. All personnel data is fake lab data (`Boss, Det. Sarah Jenkins, @forensics.org`). Regenerate a fresh export immediately before final submission if a more current sample is preferred – the freshest export is the most defensible one in the interview.

```
Log ID,User Name,User Email,Action Type,Target Entity,Entry Hash (SHA-256),Previous Entry Hash,Timestamp
84,Boss,admin@forensics.org,generate_case_final_report,Report #8,
      bbbe31e8a15561c5ce42dd82130aa05d527dd63ae10344ae4dc468d4f1fa8c91,9
      c06b865fe188ec23c9818ce47fa5908b845b62be03ac5eefb6f62a5c1667216,2026-08-02 17:32:43
83,Boss,admin@forensics.org,generate_case_final_report,Report #7,9
      c06b865fe188ec23c9818ce47fa5908b845b62be03ac5eefb6f62a5c1667216,
      b3f646ee73387999657a54810bc9430515172d4cd5ce760095e8247f5589c0e4,2026-08-02 17:32:41
82,Boss,admin@forensics.org,view_case,ForensicCase #3,b3f646ee73387999657a54810bc9430515172d4cd5ce760095e8247f5589c0e4,6432
      c9e67662be3dbcb72a717f007a0c439edc0fbba31b53a7d37b9946df1f9a,2026-08-02 17:32:39
81,Boss,admin@forensics.org,generate_case_final_report,Report #6,6432
      c9e67662be3dbcb72a717f007a0c439edc0fbba31b53a7d37b9946df1f9a,34
      eeb72d66955237866ce7299de26d7f3160bed27c41755f53b004e9658c1869,2026-08-02 17:21:42
80,Boss,admin@forensics.org,view_case,ForensicCase #3,34eeb72d66955237866ce7299de26d7f3160bed27c41755f53b004e9658c1869,
      dab54c4bc75d93d009e04dae84112acddeb3a54b3256de2c569615df2ea7da4b,2026-08-02 17:21:39
79,Boss,admin@forensics.org,view_case,ForensicCase #3,dab54c4bc75d93d009e04dae84112acddeb3a54b3256de2c569615df2ea7da4b,0530
      e9ef71c6001bcfce7fdc3fe6ee7b0700116d38ae97b79967d8bd325377fd,2026-08-02 17:20:12
78,Boss,admin@forensics.org,view_evidence,EvidenceItem #2,0530e9ef71c6001bcfce7fdc3fe6ee7b0700116d38ae97b79967d8bd325377fd,0384
      e1189846f937309062853ac0179cbd49ac47cfa8802a60b103ae9f9ec9e7,2026-08-02 17:19:44
77,Boss,admin@forensics.org,two_factor_login_success,User #2,0384e1189846f937309062853ac0179cbd49ac47cfa8802a60b103ae9f9ec9e7
      ,9966f2e27af461959ded88dafea7c34032ddb878649f9eccabbb14aea6a2272e,2026-08-02 17:16:18
76,Auditor Robert Vance,reviewer@forensics.org,view_case,ForensicCase #3,9966
      f2e27af461959ded88dafea7c34032ddb878649f9eccabbb14aea6a2272e,98791
      c32d640a7ebfd7f4a3b42497c922de5510798e3d08f192b1f98cc77127e,2026-07-31 10:37:58
75,Analyst Michael Chen,investigator@forensics.org,view_case,ForensicCase #3,98791
      c32d640a7ebfd7f4a3b42497c922de5510798e3d08f192b1f98cc77127e,1
      b257502f185b89caf6f662a193c793ed046c0e5f3b315c419868e919746fe72,2026-07-31 10:10:51
74,Boss,admin@forensics.org,view_case,ForensicCase #3,1b257502f185b89caf6f662a193c793ed046c0e5f3b315c419868e919746fe72,
      ac227c13dd6618a8dc89cfcd85ca4b534d9ddcf7c0d2f901df261bb42fe62c85,2026-07-31 10:08:49
73,Boss,admin@forensics.org,view_case,ForensicCase #3,ac227c13dd6618a8dc89cfcd85ca4b534d9ddcf7c0d2f901df261bb42fe62c85,387
      d8f6c7f36163ff6d34e65c874ab2d6dea9369877bb179d83b64a5b64e6fd9,2026-07-31 10:08:45
72,Boss,admin@forensics.org,generate_case_final_report,Report #5,387
      d8f6c7f36163ff6d34e65c874ab2d6dea9369877bb179d83b64a5b64e6fd9,111
      a1578978c963868edd6ec8c98bfe451400dc7a9a3a66499f443af7ad84392,2026-07-31 10:04:29
71,Boss,admin@forensics.org,view_case,ForensicCase #3,111a1578978c963868edd6ec8c98bfe451400dc7a9a3a66499f443af7ad84392,
      de33fd382e6356277f6589ad79a1702f387b130d9a67161f1654d968435f8954,2026-07-31 10:04:14
70,Boss,admin@forensics.org,view_evidence,EvidenceItem #2,de33fd382e6356277f6589ad79a1702f387b130d9a67161f1654d968435f8954,265
      ff6a1df30b342042585668996126f56c6ff4bb2f169727d916108f66a7f41,2026-07-31 10:03:36
69,Boss,admin@forensics.org,initiate_custody_transfer,CustodyTransfer #1,265
      ff6a1df30b342042585668996126f56c6ff4bb2f169727d916108f66a7f41,
      ee2251e8913b58cc8046e3610f242e546b9d8653b5bb4cd7fe72436ef2c603c2,2026-07-31 10:03:36
68,Boss,admin@forensics.org,view_evidence,EvidenceItem #2,ee2251e8913b58cc8046e3610f242e546b9d8653b5bb4cd7fe72436ef2c603c2,1000
      a53fe498267d6b158b0a75b0ade199b0afb766b272fbcba905c7a7b49de2,2026-07-31 10:03:01
67,Boss,admin@forensics.org,view_evidence,EvidenceItem #2,1000a53fe498267d6b158b0a75b0ade199b0afb766b272fbcba905c7a7b49de2,9443
      b0a40dc39708c2bd066d9790de2585b907ac4e994d7037755972da7e6767,2026-07-31 10:03:00
66,Boss,admin@forensics.org,view_evidence,EvidenceItem #2,9443b0a40dc39708c2bd066d9790de2585b907ac4e994d7037755972da7e6767,
      f13b5367f82e3f27e34c30769e63e767d6600602c21aa4e3d2d377dd25f433d0,2026-07-31 10:02:13
65,Boss,admin@forensics.org,view_case,ForensicCase #3,f13b5367f82e3f27e34c30769e63e767d6600602c21aa4e3d2d377dd25f433d0,
      c0b0f3dfc53b914b11cff003196f679b47f2d31d01f315f060cce11ba256805c,2026-07-31 10:02:02
64,Boss,admin@forensics.org,view_case,ForensicCase #3,c0b0f3dfc53b914b11cff003196f679b47f2d31d01f315f060cce11ba256805c,
      ecc4da9a391ec3144e03c0fe3325e34915dd3df3fef188cd7293a8d9cf1a16a0,2026-07-31 10:01:59
63,Boss,admin@forensics.org,view_evidence,EvidenceItem #2,ecc4da9a391ec3144e03c0fe3325e34915dd3df3fef188cd7293a8d9cf1a16a0
      ,96093b42d1924820a65d5fe84cd2a2d9e4494649f2fe9b9638657dc3170841ca,2026-07-31 10:01:34
62,Boss,admin@forensics.org,two_factor_login_success,User #2,96093b42d1924820a65d5fe84cd2a2d9e4494649f2fe9b9638657dc3170841ca
      ,31cf75382aaaad7e1c6654c505139ad9159f2c889a15607b61137018baf3eb37,2026-07-31 10:01:24
61,Analyst Michael Chen,investigator@forensics.org,view_case,ForensicCase #3,31
      cf75382aaaad7e1c6654c505139ad9159f2c889a15607b61137018baf3eb37,931
      b229bb50e5bf870d94221fde330cc53ce1dd49e1011d839d2b72a3a4eb2e1,2026-07-30 17:11:13
60,Analyst Michael Chen,investigator@forensics.org,view_case,ForensicCase #3,931
      b229bb50e5bf870d94221fde330cc53ce1dd49e1011d839d2b72a3a4eb2e1,95
      d8f0b1ca396d4803c9f6dd99e253e37726a18d4f26d3f00ba4cd9feaa58167,2026-07-30 17:11:07
59,Boss,admin@forensics.org,admin_update_user,User #5,95d8f0b1ca396d4803c9f6dd99e253e37726a18d4f26d3f00ba4cd9feaa58167,
      a4ec32492dec30d111c95946ec2be160b3f956814ba4d8a263defcf064fcb201,2026-07-30 17:07:30
58,Boss,admin@forensics.org,admin_update_user,User #4,a4ec32492dec30d111c95946ec2be160b3f956814ba4d8a263defcf064fcb201,
```

```
      eec545dee8381514cf454cd9f8a6d32af4d5ed746ba656ec6f5904e78c66e91c,2026-07-30 17:07:20
57,Boss,admin@forensics.org,two_factor_login_success,User #2,eec545dee8381514cf454cd9f8a6d32af4d5ed746ba656ec6f5904e78c66e91c
      ,01047a693f9289f94697f7917a08dec2c80fde593062e9c6dfc01abdc837a739,2026-07-30 17:06:55
56,Boss,admin@forensics.org,enable_two_factor_auth,User #2,01047a693f9289f94697f7917a08dec2c80fde593062e9c6dfc01abdc837a739,6
      e5d4bfbcb2eadfe24c4ba2069aa95599d231aaba1ba6fe10729ec202c0896ad,2026-07-30 17:05:55
55,Boss,admin@forensics.org,admin_system_integrity_scan,N/A,6e5d4bfbcb2eadfe24c4ba2069aa95599d231aaba1ba6fe10729ec202c0896ad
      ,857a3607ee0d389e5b115b7807b63c90f3bc8602df73638676bbd43c4ce65760,2026-07-30 16:03:22
```